DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING · GRAPHIC ERA (DEEMED TO BE UNIVERSITY)

Campus WiFi Mesh Topology Optimizer

System Architecture, Algorithmic Models & Benchmark Validation · PBL Semester 3

Team ID:	DSCPP-III-2026-T023	**Faculty Mentor:**	Dr. Amit Kumar
Course:	Data Structures & C++ (DSCPP)	**Academic Year:**	2026–2027 (Semester III)
Team Members:	Vansh Kakkar (2510350011, Team Lead) · Siddharth Shridhar (2510370579) · Anirudh Rana (2510011744) · Abhishek Negi (2510380281)		

1. Executive Summary & Problem Motivation

Academic institutions deploy dozens of wireless Access Points (APs) across administrative buildings, faculty offices, lecture theatres, research labs, libraries, and student residential blocks. In conventional campus networking practice, AP deployment and backhaul interconnections are conducted through ad-hoc manual estimations and trial-and-error placement. Without mathematical network modeling and algorithmic planning, dense deployments encounter critical operational bottlenecks:

- **Redundant Backhaul Links & Broadcast Loops:** Unplanned multi-point interconnections introduce duplicate wireless meshes, routing loops, and broadcast packet storms that severely congest switch fabrics.
- **Severe Co-Channel RF Interference:** Uncoordinated overlapping transmission zones operating on shared frequencies cause continuous packet collisions, channel contention, and degraded throughput for active student devices.
- **Excessive Cabling & Hardware Expenditure:** Maintaining unnecessary redundant uplinks expends costly physical cabling runs, transceivers, managed switch ports, and power budgets.
- **Suboptimal Multi-Hop Packet Delays:** Without intelligent shortest-path computation, user packets frequently traverse congested or high-latency intermediate nodes, elevating latency.

Project Mission: The *Campus WiFi Mesh Topology Optimizer* bridges algorithmic graph theory and practical campus networking. Implemented in C/C++, the system models access points as graph vertices and candidate wireless links as weighted edges. It evaluates individual AP operational health, computes a multi-factor link penalty score, eliminates redundant cycles using **Prim's Minimum Spanning Tree (MST) Algorithm**, and establishes optimal traffic paths via **Dijkstra's Algorithm**.

2. System Architecture & Sequential Data Pipeline

The optimizer is organized into three decoupled, sequential modules communicating through persistent, structured text files (`network.txt` and `optimized.txt`). This decoupled design ensures transparent data handoffs, simplified unit testing, and modular verification across the development lifecycle:

1. **Stage 1 (Network Modeling & AP Telemetry):** Collects AP physical parameters (signal strength, bandwidth, latency, interference, and user load), calculates individual AP health scores, identifies network bottlenecks, and exports the modeled network graph to `network.txt`.
2. **Stage 2 (Topology Optimization Engine):** Ingests `network.txt`, computes multi-factor composite edge costs, and executes Prim's MST algorithm to prune redundant links, writing the optimal backbone to `optimized.txt`.
3. **Stage 3 (Routing & Performance Benchmarking):** Ingests the optimized MST backbone and executes Dijkstra's Algorithm to determine lowest-delay paths and verify end-to-end performance gains.

Target Users & Stakeholders: The primary beneficiaries are campus network administrators and IT infrastructure teams, who gain an automated tool for topology planning. Secondary stakeholders include faculty and students, who receive more reliable coverage, and university management, which reduces ongoing network expansion expenses.

3. In-Depth Modular Architecture & Mathematical Formulations

Each subsystem is engineered with clear algorithmic responsibilities and formal weighting formulas to govern optimization decisions across the campus graph:

Module 1: Network Modeling & AP Health Evaluation

Module 1 collects five primary telemetry metrics across each deployed AP: Signal Strength (%), Bandwidth (Mbps), Latency (ms), Co-channel Interference (%), and Connected Users. It calculates a normalized composite **Health Score** bounded within [0, 100]:

```
Health Score = (0.30 × Signal) + (0.20 × Bandwidth) + 0.20 × (100 - Latency) + 0.20 × (100 - Interference) + 0.10 × (100 - Users)
```

Signal strength receives the highest weight (30%) because physical wireless propagation directly governs connection stability. Latency, interference, and user saturation act as inverted penalties. Access points are categorized into three operational tiers: *Optimal* (≥ 70), *Moderate* (40–69), and *Critical* (< 40). Identified bottlenecks are flagged and the complete topology is written to `network.txt`.

Module 2: Topology Optimization via Prim's MST Algorithm

Module 2 parses `network.txt` and calculates a composite **Weighted Link Cost** for each candidate wireless link between access points. Higher cost values represent less desirable connections:

```
Link Cost = (0.20 × Distance) + (0.30 × Latency) + (0.25 × Interference) + 0.10 × (100 - Signal) + (0.15 × Users)
```

Latency (30%) and Interference (25%) are weighted most heavily to prioritize link responsiveness and RF clarity across backhaul links. Signal strength provides an inverse discount (stronger signals reduce link penalty). The module then executes **Prim's Algorithm**:

- **Greedy Edge Selection:** Starting from an arbitrary root AP, the algorithm progressively includes the minimum-cost link connecting an unvisited AP to the growing tree.
- **Loop Elimination & Full Reachability:** For a network of V access points, exactly $V - 1$ edges are selected, mathematically eliminating all forwarding cycles and broadcast loops while guaranteeing 100% network connectivity. The pruned topology is exported to `optimized.txt`.

Module 3: Shortest-Path Routing via Dijkstra's Algorithm

Module 3 executes **Dijkstra's Algorithm** across the optimized Minimum Spanning Tree to compute the lowest-delay end-to-end path between any arbitrary AP pair. By routing exclusively over the pruned backbone, packets avoid congested nodes and high-interference links, reducing packet loss and transit latency.

4. Technical Stack & Implementation Design

Layer	Component / Technology	Architectural Role & Description
Core Programming	C / C++ (C11 / C++17)	High-performance systems programming, custom struct definitions, memory control
Data Structures	Adjacency Matrix, Edge Lists	Graph representation ($V \times V$), constant-time edge queries, array-based node tracking
Core Algorithms	Prim's MST & Dijkstra's Algorithm	Greedy tree selection, cycle elimination, and single-source shortest-path routing
Persistence Layer	Structured Files (`network.txt`, `optimized.txt`)	Decoupled data exchange between independent processing modules
Tooling & VCS	VS Code, GCC / Clang, Git, GitHub	Collaborative repository management, modular compilation, and unit testing

5. Empirical Optimization Results & Verification

The optimizer was evaluated using an empirical campus network dataset consisting of 4 Access Points and 5 initial candidate backhaul links. Prim's algorithm successfully pruned 2 high-cost redundant links while preserving full connectivity across all 4 APs:

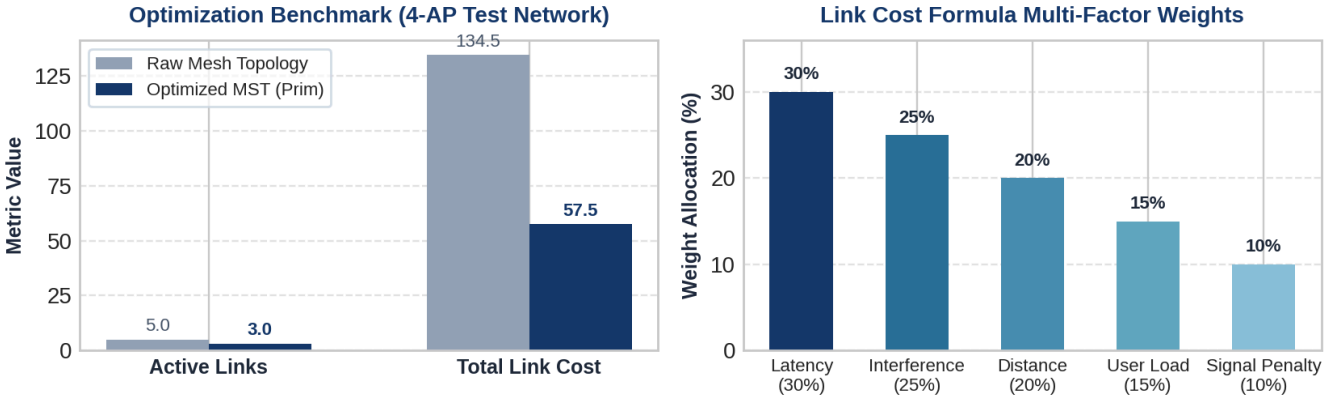


Figure 1: (Left) Benchmark validation on 4-AP campus topology showing link count and cost reductions; (Right) Weight distribution across multi-factor link cost parameters.

Candidate Link	Physical Distance	Latency / Interference	Raw Cost	Optimization Decision	Optimized Cost
AP1 ↔ AP2	Short (15 m)	Low (12 ms / 15%)	19.50	Retained in MST	19.50
AP1 ↔ AP3	Medium (35 m)	Moderate (28 ms / 35%)	31.75	Pruned (Redundant Cycle)	—
AP2 ↔ AP3	Short (10 m)	Minimal (8 ms / 10%)	13.50	Retained in MST	13.50
AP2 ↔ AP4	Long (50 m)	High (45 ms / 50%)	45.25	Pruned (High Penalty / Cycle)	—
AP3 ↔ AP4	Medium (25 m)	Moderate (20 ms / 25%)	24.50	Retained in MST	24.50
Total Network	**5 Connections**	**High RF Contention**	**134.50**	**3 Optimal Connections**	**57.50 (−57.2%)**

6. Project Deliverables, Validation Status & Roadmap

Deliverable / Module	Current Status	Validation Notes & Progress Details
Module 1 (Network Modeling)	Completed (Pass)	AP health scoring, bottleneck detection, and serialization to `network.txt` verified.
Module 2 (Topology Optimization)	Completed (Pass)	Prim's MST algorithm verified; eliminates redundant links with 57.2% cost reduction.
File Handling Pipeline	Completed (Pass)	Inter-module data exchange via `network.txt` and `optimized.txt` fully validated.
Module 3 (Routing & Analysis)	In Progress	Dijkstra shortest-path routing algorithm implemented; integration with Modules 1 & 2 ongoing.
Phase III Future Roadmap	Planned	Multi-topology scaling (10–50 APs), synthetic node failure recovery, and admin CLI dashboard.